ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор		Т.М. Татарникова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Моделирование базовой случайной величины

по курсу: Моделирование систем

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4321		И.А. Лаврешин
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Построить датчик базовой случайной величины по заданному алгоритму и выполнить тестирование датчика на соответствие основным свойствам базовой случайной величины.

2 Задание и исходные данные

В соответствии с вариантом 9, представленным в таблице 1, необходимо построить датчик БСВ с периодом T>500, оценить математическое ожидание и дисперсию псевдослучайных значений z_i и сравнить их с теоретическими значениями M и D. Проверить датчик БСВ на равномерность и построить гистограмму распределения относительных частот $p_1,\ldots, p_k$ на K отрезках интервала [0,1]. Проверить датчик БСВ на независимость, определяя коэффициент корреляции для разных значений s и T. Построить в одном графическом окне графики зависимости R от T для s=2, s=5, s=10.

Таблица 1 – Исходные данные

№ п/п	Задание
9.	**Линейная смешанная формула** $A_i = (k_1A_{i-1}+\ldots+k_rA_{i-r} + C) \bmod m$, i=1,2, …, $z_i = A_i/m$, где r – порядок рекурсии, A0, A1, …, Ar+1 – стартовые значения, k1, …, k_r – множители. Длина периода $T<m^r$.

3 Ход выполнения задания

Основой для выполнения данной работы послужило создание программного модуля, генерирующего поток псевдослучайных чисел. Вычислительный процесс базируется на рекуррентном методе третьего порядка, который также известен как линейная смешанная формула. Этот выбор обусловлен заданием варианта № 9 из соответствующих методических материалов. Формирование каждого нового элемента последовательности целых чисел Ak осуществляется строго в соответствии со следующим законом:

$$A_k = (1664525 \cdot A_{k-1} + 1013904223 \cdot A_{k-2} + 1103515245 \cdot A_{k-3} + 12345) \bmod 65536,$$

где задействованные числовые коэффициенты строго фиксированы, а модуль m, равный 2 в 16-й степени (65536), определяет максимальный предел генерируемых целочисленных значений.

Для инициализации алгоритма необходимо задать три опорных значения A0, A1 и A2. В рамках данной работы они приняты равными 123, 456 и 789 соответственно. Именно эти величины определяют уникальность порождаемой числовой последовательности. С целью получения стандартизированной случайной величины z, принадлежащей интервалу [0, 1), каждое вычисленное целое Ak подвергается нормализации путем деления на модуль m: zk = Ak / 65536. Используя описанный подход, конструируется выборка требуемого объема n, состоящая из элементов z1, z2, ..., zn.

Сформированный массив данных подвергается всестороннему статистическому исследованию. Первоначально рассчитываются выборочные характеристики — среднее значение и показатель вариации (дисперсия). Полученные эмпирические оценки сопоставляются с эталонными параметрами теоретической модели равномерного распределения на отрезке от 0 до 1, где математическое ожидание составляет 0,5, а дисперсия — 1/12. На следующей стадии исследуемый диапазон [0, 1] дробится на K равных промежутков. Для каждого из них вычисляется частота попадания элементов

выборки (в относительном выражении). По этим данным визуализируется гистограмма, наглядно демонстрирующая, насколько эмпирическое распределение приближается к теоретически равномерному. Помимо анализа распределения, проверяется гипотеза о независимости элементов. Для этого рассчитывается корреляционная связь между членами последовательности, разнесенными на фиксированный шаг s. Для ряда значений шага, таких как 2, 5 и 10, строятся кривые, отражающие динамику изменения вычисленного коэффициента корреляции R по мере увеличения объема выборки T, что позволяет судить об изменении взаимозависимости элементов при накоплении данных.

3.1 Сравнение числовых значений

По итогам проведенного эксперимента, представленного на рисунке 1, можно утверждать, что разработанный генератор на основе линейной смешанной формулы демонстрирует высокую степень соответствия эмпирических параметров эталонным характеристикам равномерного распределения. Вычисленное выборочное среднее M^z оказалось весьма близким к теоретическому значению 0,5, в то время как оценка дисперсии D^z незначительно отклоняется от величины 1/12 (приблизительно 0,0833). Наблюдаемые малые расхождения находятся в пределах допустимой погрешности и обусловлены ограниченным объемом анализируемой выборки, а также действием случайных факторов. Следовательно, на основании анализа численных характеристик можно заключить, что представленный алгоритм успешно выполняет функцию формирования базовой случайной величины, подчиняющейся закону равномерного распределения на отрезке [0, 1]..

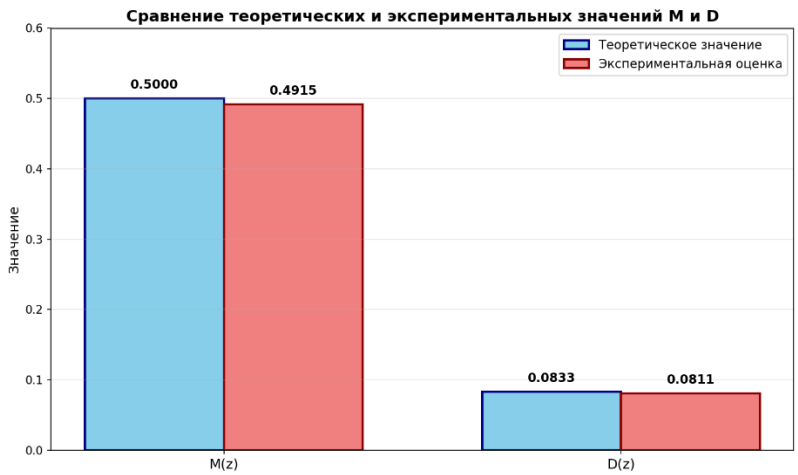

Рисунок 1 – Сравнение теоретические и экспериментальных значений M и D

3.2 Равномерность

На рисунке 2 представлена гистограмма, отражающая распределение относительных частот по K равным промежуткам, на которые разбит интервал [0, 1]. Как видно из диаграммы, наблюдается равномерное колебание частот вокруг эталонной величины 1/K. Отсутствие ярко выраженных систематических провалов или всплесков, а также тот факт, что большинство столбцов гистограммы расположены вблизи теоретической отметки, подтверждает гипотезу о соответствии сгенерированной последовательности закону равномерного распределения. Имеющиеся незначительные отклонения значений по отдельным интервалам не выходят за рамки допустимых статистических погрешностей и связаны с ограниченным размером исследованной выборки.

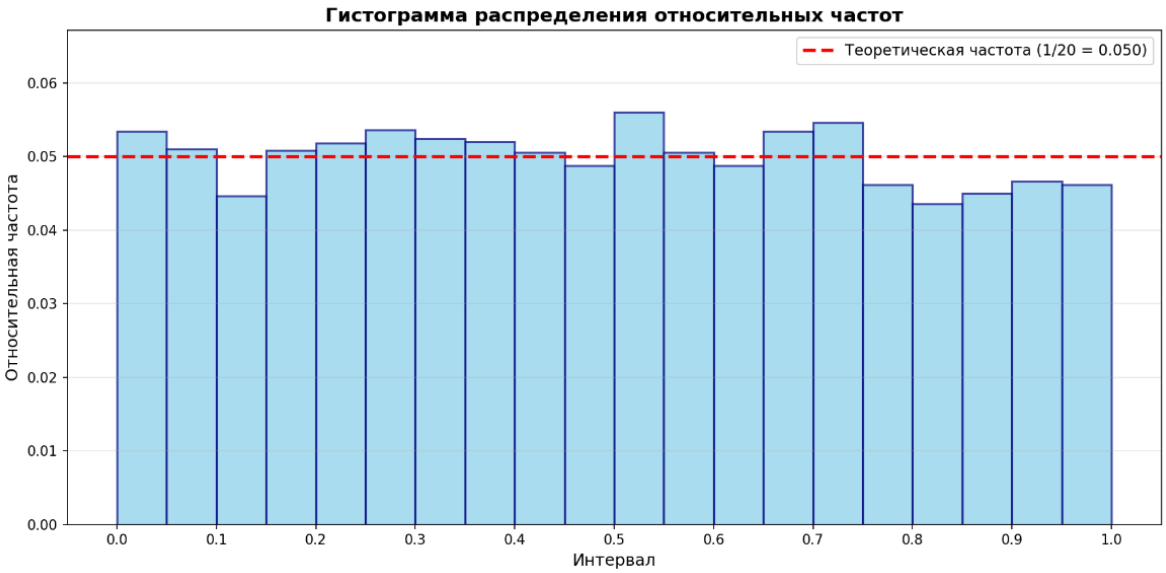

Рисунок 2 – Гистограмма распределения относительных частот

3.3 Независимость

Для оценки степени независимости сгенерированных псевдослучайных величин был проведен корреляционный анализ. С этой целью вычислялся коэффициент корреляции для пар элементов, сдвинутых друг относительно друга на заданное число позиций s, то есть для сочетаний (z_i, z_{i+s}). Используя разработанный вычислительный модуль, были получены зависимости коэффициента R от объема обработанной выборки T для трех значений шага: s = 2, s = 5 и s = 10 (графики приведены на Рисунке 3). Анализ представленных кривых показывает, что по мере роста T величина коэффициента корреляции стремится к нулю, испытывая незначительные флуктуации вблизи этого значения. Подобная динамика свидетельствует об отсутствии существенной линейной взаимосвязи между членами последовательности на исследованных временных сдвигах. Таким образом, можно утверждать, что при достаточно больших размерах выборки генерируемые числа проявляют свойства статистической независимости.

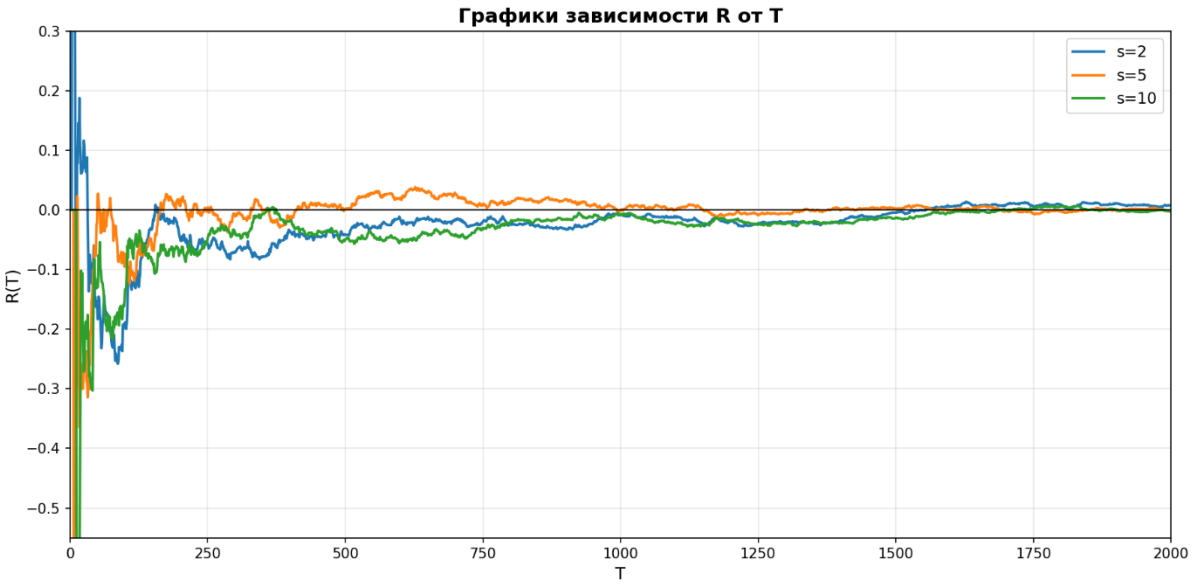

Рисунок 3 – График зависимости R от T

4 Вывод

Разработанный в ходе выполнения работы программный генератор псевдослучайных чисел, основанный на линейной рекуррентной формуле третьего порядка (вариант № 9), продемонстрировал свою эффективность и пригодность для практического использования. Использованные числовые параметры ($m = 65536$, $k_1 = 1664525$, $k_2 = 1013904223$, $k_3 = 1103515245$, $C = 12345$) совместно с начальными условиями ($A_0 = 123$, $A_1 = 456$, $A_2 = 789$) обеспечивают корректную работу алгоритма.

Вычислительные эксперименты подтвердили высокую точность моделирования базовой случайной величины: рассчитанные по выборке оценки математического ожидания и дисперсии незначительно отклоняются от эталонных параметров равномерного закона распределения на отрезке [0, 1]. Это свидетельствует о правильности программной реализации и адекватности выбранных констант. Визуальный анализ гистограммы относительных частот показывает равномерное распределение данных по интервалам; наблюдаемые малые отклонения от теоретической прямой 1/K находятся в пределах статистической погрешности и не влияют на общую картину, подтверждая хорошие показатели равномерности генерируемой последовательности.

Дополнительно проведенное исследование корреляционных связей для различных значений шага сдвига s выявило, что вычисленные коэффициенты близки к нулевому значению. Графики, отображающие зависимость коэффициента корреляции от объема выборки, наглядно демонстрируют тенденцию к затуханию корреляций по мере увеличения T. Совокупность полученных результатов позволяет утверждать, что созданный датчик базовых случайных величин обладает необходимым набором статистических свойств и может быть рекомендован к применению в рамках имитационного моделирования согласно требованиям лабораторного практикума.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
from __future__ import annotations
import argparse
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
class LinearCongruentialGeneratorR3:
    def __init__(
        self,
        A0: int = 123,
        A1: int = 456,
        A2: int = 789,
        m: int = 2**16,
        k1: int = 1664525,
        k2: int = 1013904223,
        k3: int = 1103515245,
        C: int = 12345,
    ) -> None:
        self.m = m
        self.k1 = k1
        self.k2 = k2
        self.k3 = k3
        self.C = C
        self.A = [A0 % m, A1 % m, A2 % m]
        self.current_index = 2
    def next(self) -> float:
        i = self.current_index
        new_A = (
            self.k1 * self.A[i]
```

```
            + self.k2 * self.A[i - 1]
            + self.k3 * self.A[i - 2]
            + self.C
        ) % self.m
        self.A.append(new_A)
        self.current_index += 1
        return new_A / self.m
    def generate_sequence(self, n: int) -> np.ndarray:
        sequence = [self.next() for _ in range(n)]
        return np.array(sequence, dtype=float)
def calculate_correlation(sequence: np.ndarray, s: int) -> float:
    """Расчёт обычного коэффициента корреляции Пирсона между z_i и z_{i+s}."""
    T = len(sequence)
    if T <= s + 1:
        return 0.0
    x = sequence[:-s]
    y = sequence[s:]
    if x.std(ddof=1) == 0 or y.std(ddof=1) == 0:
        return 0.0
    r = np.corrcoef(x, y)[0, 1]
    return float(r)
def _ensure_outdir(path: str) -> Path:
    outdir = Path(path).expanduser().resolve()
    outdir.mkdir(parents=True, exist_ok=True)
    return outdir
def main() -> int:
    ap = argparse.ArgumentParser(
        description="ЛР1, вариант 9: линейная смешанная формула (r=3)"
    )
```

```
ap.add_argument(
    "--n",
    type=int,
    default=5000,
    help="объём выборки T_total (по умолчанию 5000)",
)
ap.add_argument(
    "--k",
    type=int,
    default=20,
    help="число интервалов/бинов на [0,1) для гистограммы (по умолчанию 20)",
)
ap.add_argument(
    "--t-min",
    type=int,
    default=100,
    help="минимальное T для графика зависимости R от T (по умолчанию 100)",
)
ap.add_argument(
    "--t-max",
    type=int,
    default=2000,
    help="максимальное T для графика зависимости R от T (по умолчанию 2000)",
)
ap.add_argument(
    "--t-step",
    type=int,
```

```
        default=100,
        help="шаг по T для графика зависимости R от T (по умолчанию 100)",
    )
    ap.add_argument(
        "--s-list",
        type=str,
        default="2,5,10",
        help='список значений s для корреляции, формат "2,5,10"',
    )
    ap.add_argument(
        "--outdir",
        type=str,
        default=str(Path(__file__).with_name("plots_variant9")),
        help="директория для сохранения PNG",
    )
    ap.add_argument(
        "--show",
        action="store_true",
        help="показывать окна с графиками (по умолчанию только сохраняет PNG)",
    )
    args = ap.parse_args()
    outdir = _ensure_outdir(args.outdir)
    print("=" * 70)
    print("ЛАБОРАТОРНАЯ РАБОТА №1: ПОСТРОЕНИЕ И ТЕСТИРОВАНИЕ ДАТЧИКА БСВ")
    print("=" * 70)
    print("Вариант 9: Линейная смешанная формула (r = 3)")
    print("Параметры датчика:")
    print("  r = 3 (порядок рекурсии)")
```

```
print("  m = 2^16 = 65536")
print("  k1 = 1664525, k2 = 1013904223, k3 = 1103515245")
print("  C = 12345")
print("  A0 = 123, A1 = 456, A2 = 789")
print("-" * 70)
print("\nГенерация последовательности...")
rng = LinearCongruentialGeneratorR3()
T_total = int(args.n)
z = rng.generate_sequence(T_total)
print(f"Сгенерировано {T_total} псевдослучайных чисел")
print(f"Первые 10 значений: {[round(x, 4) for x in z[:10]]}")
M_theoretical = 0.5
D_theoretical = 1.0 / 12.0
M_empirical = float(np.mean(z))
D_empirical = float(np.var(z, ddof=1))
plt.figure(figsize=(10, 6))
categories = ["M(z)", "D(z)"]
theoretical_values = [M_theoretical, D_theoretical]
empirical_values = [M_empirical, D_empirical]
x = np.arange(len(categories))
width = 0.35
bars1 = plt.bar(
    x - width / 2,
    theoretical_values,
    width,
    label="Теоретическое значение",
    color="skyblue",
    edgecolor="navy",
    linewidth=2,
)
```

```
bars2 = plt.bar(
    x + width / 2,
    empirical_values,
    width,
    label="Экспериментальная оценка",
    color="lightcoral",
    edgecolor="darkred",
    linewidth=2,
)
for bar in bars1:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2.0,
        height + 0.01,
        f"{height:.4f}",
        ha="center",
        va="bottom",
        fontsize=11,
        fontweight="bold",
    )
for bar in bars2:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2.0,
        height + 0.01,
        f"{height:.4f}",
        ha="center",
        va="bottom",
        fontsize=11,
        fontweight="bold",
```

```
    )

plt.ylabel("Значение", fontsize=12)
plt.title(
    "Сравнение теоретических и экспериментальных значений M и D",
    fontsize=14,
    fontweight="bold",
)
plt.xticks(x, categories, fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3, axis="y")
plt.ylim(0, 0.6)
plt.tight_layout()
md_path = outdir / "01_mean_variance_comparison.png"
plt.savefig(md_path, dpi=150)
if args.show:
    plt.show()
else:
    plt.close()
K = int(args.k)
plt.figure(figsize=(12, 6))
counts, bins = np.histogram(z, bins=K, range=(0.0, 1.0))
relative_frequencies = counts / float(T_total)
plt.bar(
    bins[:-1],
    relative_frequencies,
    width=bins[1] - bins[0],
    alpha=0.7,
    color="skyblue",
    edgecolor="navy",
```

```
    linewidth=1.5,
    align="edge",
)
theoretical_relative_freq = 1.0 / K
plt.axhline(
    y=theoretical_relative_freq,
    color="red",
    linestyle="--",
    linewidth=2.5,
    label=f"Теоретическая частота (1/{K} = {1.0 / K:.3f})",
)

plt.xlabel("Интервал", fontsize=12)
plt.ylabel("Относительная частота", fontsize=12)
plt.title(
    "Гистограмма распределения относительных частот",
    fontsize=14,
    fontweight="bold",
)
plt.grid(True, alpha=0.3, axis="y")
plt.legend(fontsize=11)
plt.xticks(np.linspace(0.0, 1.0, 11), [f"{i/10:.1f}" for i in range(11)])
plt.ylim(0, max(relative_frequencies) * 1.2 if len(relative_frequencies) else 0.15)

plt.tight_layout()
hist_path = outdir / "02_histogram.png"
plt.savefig(hist_path, dpi=150)
if args.show:
    plt.show()
else:
```

```
    plt.close()
T_values = np.arange(1, 2001)  # T = 1..2000
s_values = [2, 5, 10]
colors = ["C0", "C1", "C2"]
plt.figure(figsize=(12, 6))
for i, s in enumerate(s_values):
    R_values: list[float] = []
    for T_current in T_values:
        R = calculate_correlation(z[:T_current], s)
        R_values.append(R)
    plt.plot(
        T_values,
        R_values,
        color=colors[i],
        linewidth=1.8,
        label=f"s={s}",
    )
plt.axhline(y=0, color="black", linestyle="-", linewidth=1.0)
plt.xlabel("T", fontsize=12)
plt.ylabel("R(T)", fontsize=12)
plt.title("Графики зависимости R от T", fontsize=14, fontweight="bold")
plt.grid(True, alpha=0.3)
plt.legend(fontsize=11, loc="upper right")
# Диапазоны по осям, близкие к примеру
plt.ylim(-0.55, 0.3)
plt.xlim(0, 2000)
plt.xticks([0, 250, 500, 750, 1000, 1250, 1500, 1750, 2000])
plt.tight_layout()
corr_path = outdir / "03_correlation.png"
plt.savefig(corr_path, dpi=150)
```

```
    if args.show:
        plt.show()
    else:
        plt.close()
    print("\n" + "=" * 70)
    print("ВЫВОДЫ О РЕЗУЛЬТАТАХ МОДЕЛИРОВАНИЯ БСВ")
    print("=" * 70)
    print("1. Математическое ожидание и дисперсия:")
    print(f"   - M(z) = {M_empirical:.4f} (теор. {M_theoretical:.4f})")
    print(f"   - D(z) = {D_empirical:.4f} (теор. {D_theoretical:.4f})")
    print("\n2. Равномерность распределения подтверждается гистограммой")
    print("\n3. Статистическая независимость:")
    ci_95 = 1.96 / np.sqrt(2000)
    print(f"   95% доверительный интервал при T=2000: ±{ci_95:.4f}")
    all_in_ci = True
    for s in s_values:
        R_Tmax = calculate_correlation(z[:2000], s)
        print(f"   s={s}: R = {R_Tmax:.4f}")
        if abs(R_Tmax) > ci_95:
            all_in_ci = False
    if all_in_ci:
        print("\n   ✓ Все значения R(T_max) находятся внутри доверительного
интервала")
        print("   ✓ Корреляция статистически незначима")
        print("   ✓ Элементы последовательности можно считать независимыми")
    else:
        print(
            "\n   Обнаружена статистически значимая корреляция "
            "для некоторых значений s")
```

```
    return 0
if __name__ == "__main__":
    raise SystemExit(main())
```